

Upgrade Customized Components

When you customize built-in React components in your storefront and Salesforce later releases updates to those components, you need a strategy to adopt the improvements while preserving your customizations. Either merge updates without conflict, or maintain separate copies and use Vite build-time aliasing. Select the approach that best fits your team's workflow and the extent of your customizations.

Method 1: Merge Updates Directly

If you've modified a component's source code and want to adopt Salesforce updates to that component, merge the changes. For straightforward cases with minimal code conflicts, a supervised Git merge can be sufficient. Carefully review all changes during the merge process to ensure your customizations are preserved while incorporating Salesforce improvements.

When to Use This Method

- Your customizations are minimal.
- Code conflicts are straightforward to resolve.
- You want to maintain a single version of each component.
- You're comfortable with Git merge workflows.

Trade-offs

Advantages:

- Automatically receive Salesforce improvements and bug fixes.



files to manage.

Disadvantages:

- Potential merge conflicts when Salesforce releases updates.
- Requires immediate conflict resolution to adopt updates.
- May require rework of customizations if Salesforce makes significant structural changes.
- Less predictable upgrade process.

Method 2: Maintain Separate Component Copies

To reduce merge complexity, maintain customized copies of components in a separate directory and use Vite's build-time aliasing to override the built-in implementations. This approach prevents Git conflicts and simplifies adoption of upstream template updates.

When to Use This Method

- You have extensive customizations.
- Merge conflicts are complex or frequent.
- You prefer explicit control over when to adopt Salesforce updates.
- You want to minimize disruption during Storefront Next releases.

Implementation Steps

1. Create a separate directory for your customized components (for example, `/customizations`).
2. Copy the component you want to customize to this directory.
3. Modify your copy as needed.
4. Configure Vite to use your custom component instead of the original.

Vite Configuration Example



```
1 // vite.config.ts
2 import { defineConfig } from "vite"
3 import { reactRouter } from "@react-router"
4 import tailwindcss from "tailwindcss"
5 import tsconfigPaths from "vite-tsconfig-paths"
6 import storefrontNextPlugin from "storefront-next"
7 import path from "path";
8
9 export default defineConfig({
10   plugins: [reactRouter(), tailwindcss(), storefrontNextPlugin()],
11   resolve: {
12     alias: {
13       // Replace the original component path with the custom path
14       "@components/ProductCard": path.resolve(__dirname, "components/ProductCard.tsx"),
15     },
16   },
17 });
```

For more information about Vite's aliasing capabilities, see the [Vite resolve.alias documentation](#).

Trade-offs

Advantages:

- Eliminates Git merge conflicts when Salesforce updates components.
- Original components remain untouched.
- Clear separation between out-of-the-box and customized code.

Disadvantages:

- You don't automatically receive Salesforce improvements.
- You must manually merge Salesforce updates into your customized copies.
- Requires maintaining awareness of upstream changes.

Note About UI Extensions

The Storefront Next extension mechanism and UI target functionality are designed for independent software vendors (ISVs) and partners building reusable solutions. These features are not intended for customizing individual templates in your storefront. Use the methods described in this guide for component customization instead.



Try for free



can improve!



DEVELOPER CENTERS

Heroku
MuleSoft
Tableau
Commerce Cloud
Lightning Design System
Einstein
Quip

POPULAR RESOURCES

Documentation
Component Library
APIs
Trailhead
Sample Apps
AppExchange

COMMUN

Trailblazer
Events and
Partner Cor
Blog
Salesforce
Salesforce

© Copyright 2026 Salesforce, Inc. [All rights reserved](#). Various trademarks held by their respective owners.
Salesforce, Inc. Salesforce Tower, 415 Mission Street, 3rd Floor, San Francisco, CA 94105, United States

[Legal](#) [Terms of Service](#) [Privacy Information](#) [Responsible Disclosure](#) [Trust](#) [Contact](#)

[Use of Cookies](#) [Cookie Preferences](#)  [Your Privacy Choices](#)